

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Relatório do Tutorial 10 - DevSecOps Pipeline de Laboratório de Engenharia de Software (LES)

Diogo Emanuel Matos Honório **2021232043**

Phase 1:



Figure 1 – First CI workflow

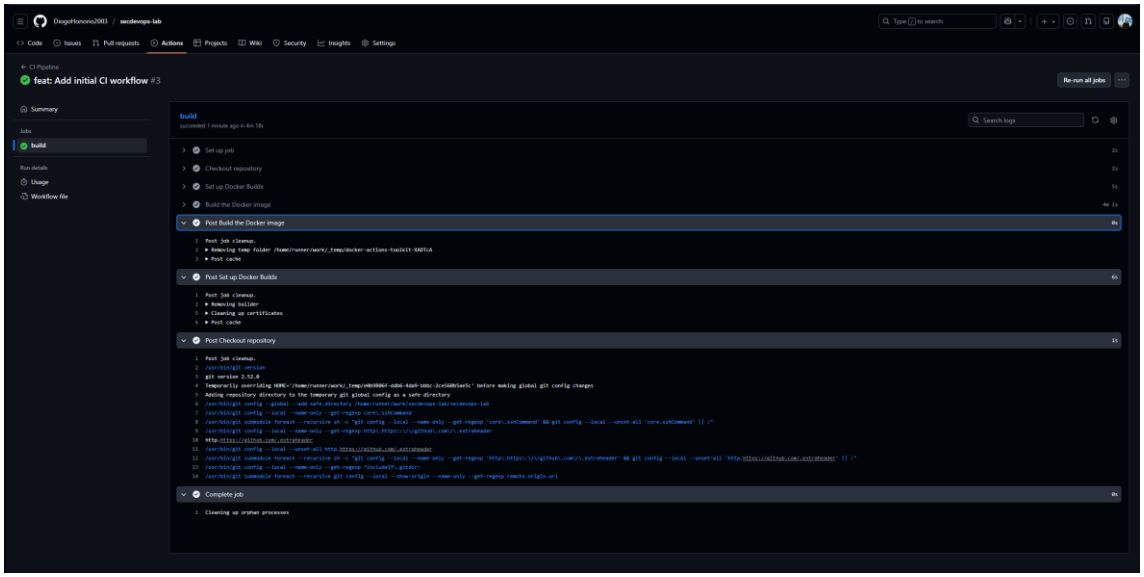


Figure 2 – First CI workflow (Details)

The first CI workflow was successfully executed, next it will be done a simple explanation about the sections in the ci.yml file:

```yaml
name: CI Pipeline

on:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build the Docker image
      uses: docker/build-push-action@v5
      with:
        context: .
        push: false
        tags: juice-shop-image:latest
```

1
2
3
4

Figure 3 – First ci.yml

1- The “name” field identifies the pipeline, determining what name is displayed in GitHub Actions interface.
2- The “on” section is responsible for defining when and where the workflow runs. In this configuration, the pipeline is triggered automatically on every push to the main branch.
3- The “jobs” section will define what runs. Here, it is where is defined to execute on GitHub's Ubuntu environment
4- The “steps” section are the sequencial actions within the job, The first one (actions/checkout) responsible for cloning the repository into the runner. The second will inicialize the Docker Buildx to optimize the image building process. And lastly it is built a Docker image using the project's Dockerfile

Phase 2:

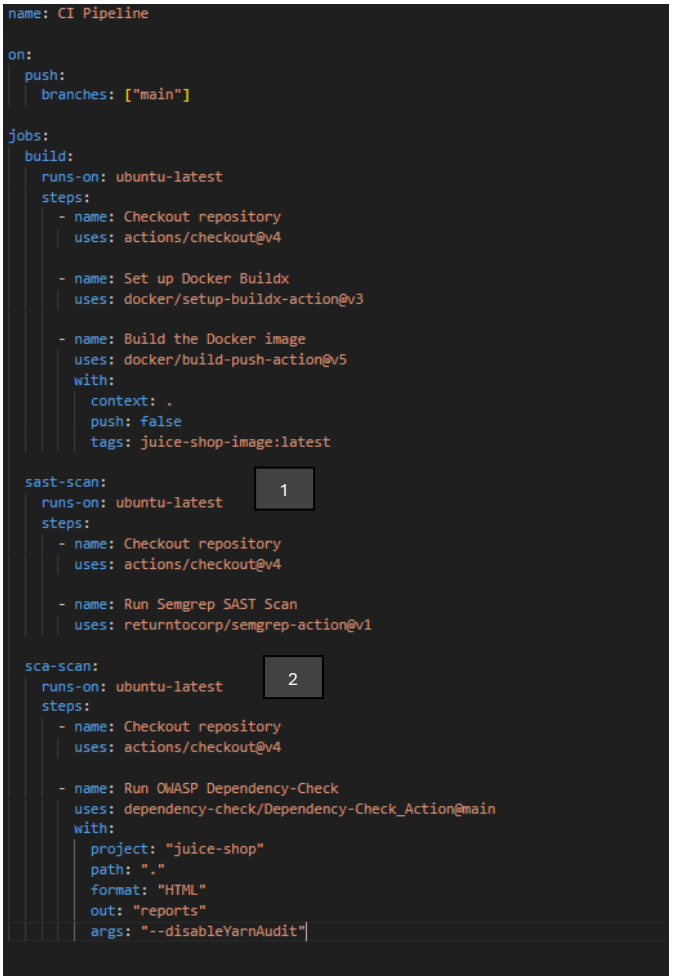


Figure 4 – Second ci.yml

1 - SAST-Scan Job: It is the job responsible for running the Semgrep, a tool that scans for security vulnerabilities and code quality issues directly in the source files. This job runs independently on an Ubuntu runner, first checking the repositor and only then running the Semgrep. This job focus on identifying security patterns and potential vulnerabilities within the codebase itself.

2 - SCA-Scan Job: This Job uses the OWASP Dependency-Check, this tool is responsible for analysing the project's third-party libraries and dependencies for known vulnerabilities. This is a job that also runs in parallel on an Ubuntu runner, and as well as the SAST-Scan Job first checks the repositor and then includes the needed configuration information for the scanning. This is crucial for identifying security risks introduced through external libraries and dependencies.

The Github knows what to find in the “uses” clauses by downloading and executing the pre-built actions of the public GitHub repositories. The clauses follow this format “owner/repository@version” that enables to point directly into reusable workflow components that GitHub automatically fetches and runs.

For example, the “uses: actions/checkout@v4” clause downloads version 4 of the official GitHub checkout action, which then clones the repository code into the runner. Another example is “uses: returntocorp/semgrep-action@v1”, which installs and executes version 1 of the Semgrep scanning tool.

After pushing the changes this error was visible:

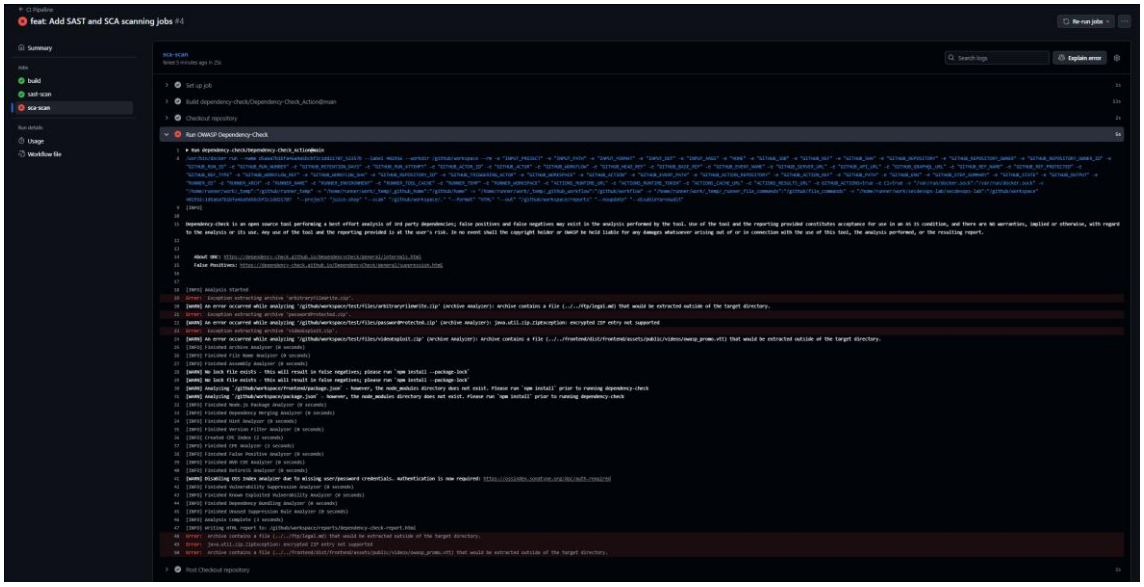


Figure 5 – Second CI workflow (Error Details)

This error indicates that the SCA tool has detected a security flaw in the dependencies of the juice-shop project. This error occurs because the tool analyzed archives from “test/files/” directory. This files have malicious paths, designed to trigger the scaning tool. Specifically, the error reveals a Path Traversal vulnerability. This error represents a scenario where an attacker would be able to overwrite system files, steal sensitive data, or compromise the entire server. Since files like “legal.md” and “owasp_promo.vtt” are exposed. Although the build of the pipeline did not stopped, this is due to the sca-scan configuration, that is merely for security checking and does not include a configuration to break the build upon detecting errors.

Phase 3:

```yaml
name: CI Pipeline

on:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and export to Docker
        uses: docker/build-push-action@v5
        with:
          context: .
          push: false
          tags: juice-shop-image:latest
          outputs: type=docker,dest=/tmp/juice-shop-image.tar

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: juice-shop-image
          path: /tmp/juice-shop-image.tar

  sast-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run Semgrep SAST Scan
        uses: returntocorp/semgrep-action@v1

  sca-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: "juice-shop"
          path: "."
          format: "HTML"
          out: "reports"
          args: "--disableYarnAudit"

  container-scan:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v4
        with:
          name: juice-shop-image
          path: /tmp

      - name: Scan container image with Trivy
        uses: crazy-max/ghaction-container-scan@v3
        with:
          tarball: /tmp/juice-shop-image.tar
```

1

2

Figure 6 – Third ci.yml

In Phase 3 the main changes were:

1 – In the “build” job it was added the “outputs” parameter, this will export the built image as a “.tar” to the runner’s “/tmp” directory. Besides that it was also added the “Upload artifact” step, it will be used to save the “.tar” file as a GitHub Actions artifact, enabling, this way, the file to other jobs in the workflow.

2- It was added the “container-scan” job, this job will only be executed once the build executes since it has a “needs: build”. After the executiopn of the build the job will download the artifact, created in the “build” job, and only then make use of the Tivy scanner to perform a vulnerability assessment on the actual Docker image that would be deployed, checking its operating system packages and application dependencies.

After the pushed changes this were the results:

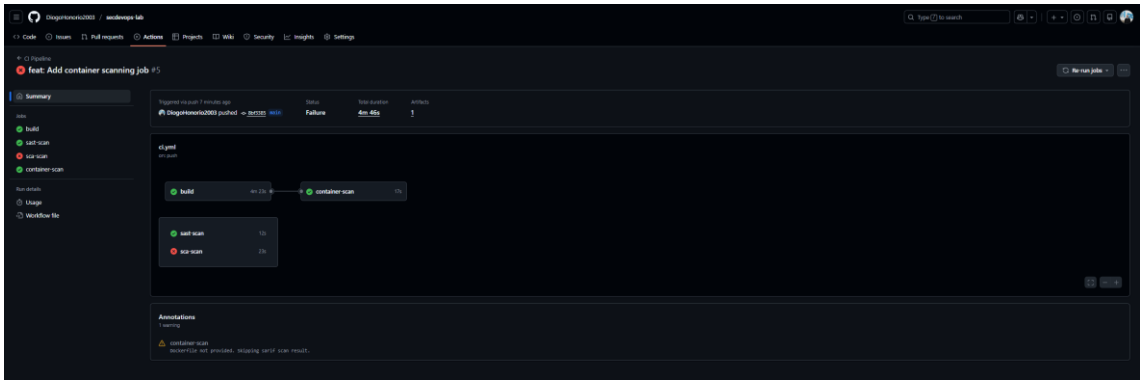


Figure 5 – Third CI workflow (Container Image Security Scanning)

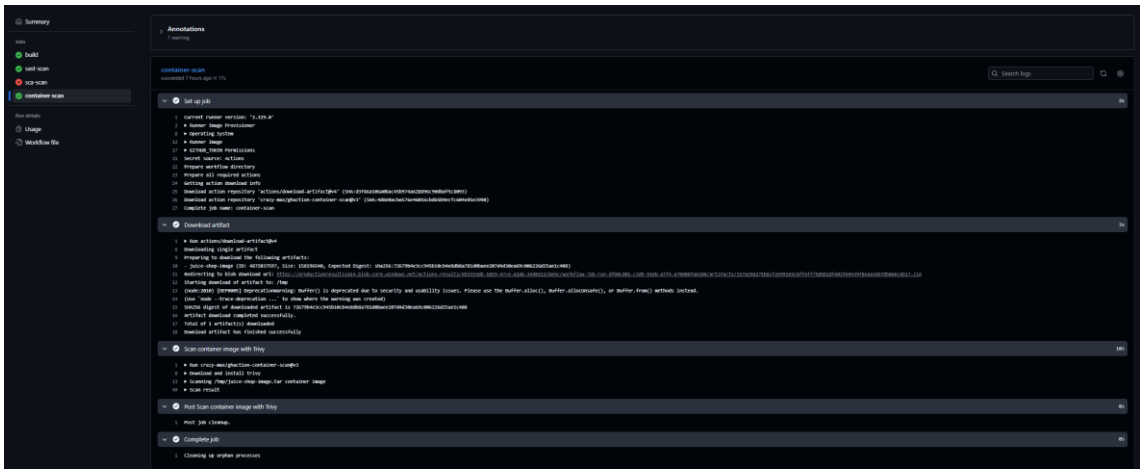
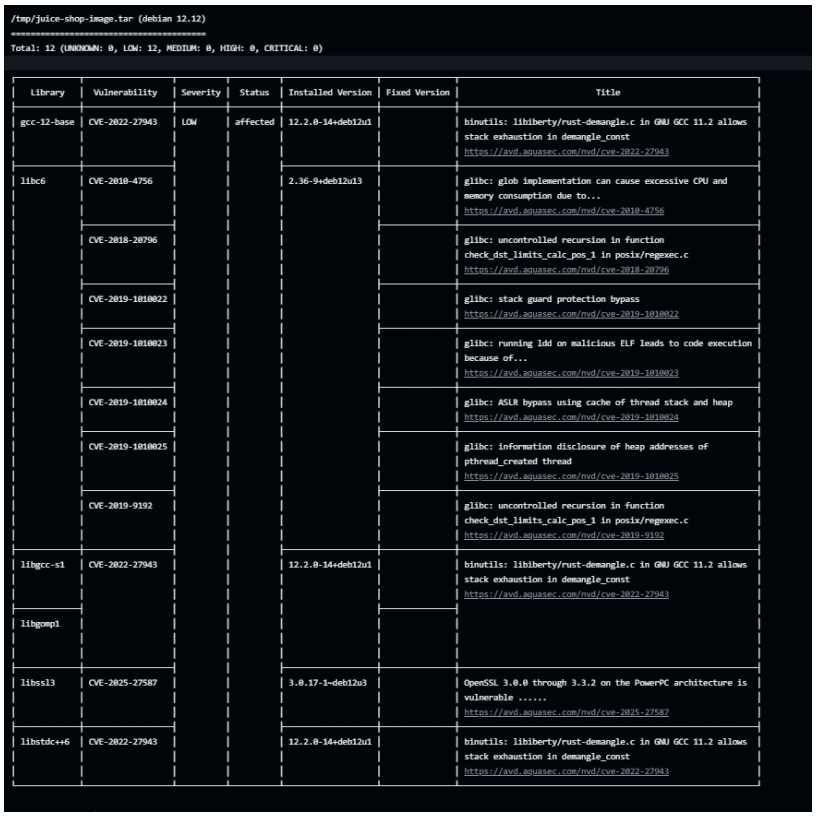


Figure 6 – Third CI workflow (Container Image Security Scanning - Details)

As shown in Figure 7, the scanner detected 12 vulnerabilities within the operating system layer of the image (/tmp/juice-shop-image.tar). Figure 8 reveals a more critical finding: 55 vulnerabilities within the application's Node.js dependencies. Furthermore, Figure 9 demonstrates that the scan exposed 4 sensitive secrets within the container: 2 private keys and 2 JWT tokens.

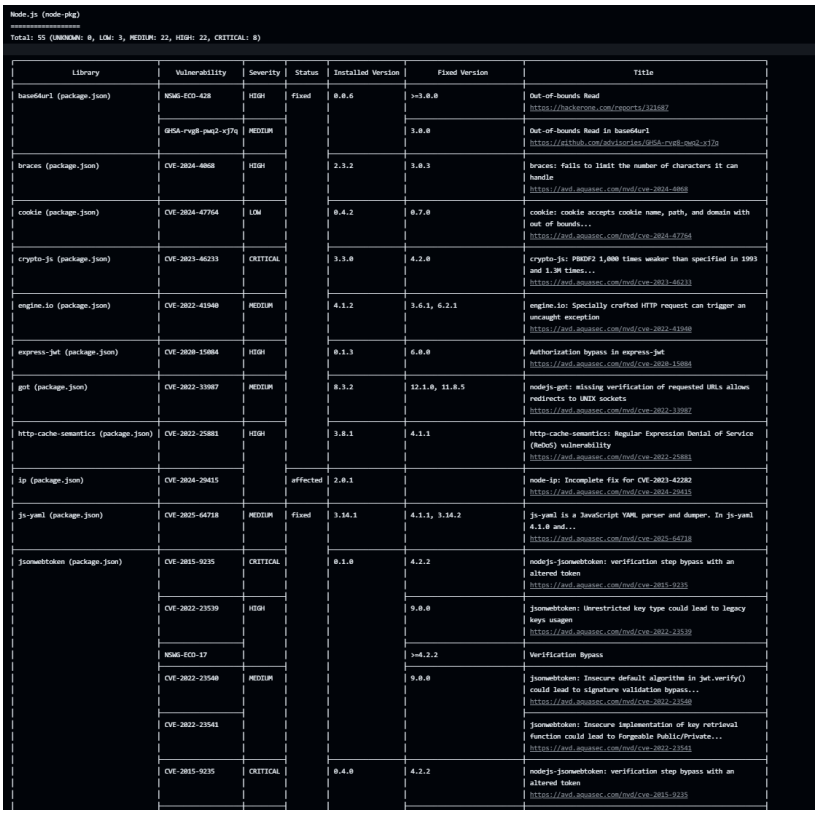


/tmp/juice-shop-image.tar (debian 12.12)

Total: 12 (UNKNOWN: 0, LOW: 12, MEDIUM: 0, HIGH: 0, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
gcc-12-base	CVE-2022-27943	LOW	affected	12.2.0-14+deb12u1		binutils: libiberty/rust-demangle.c in GNU GCC 11.2 allows stack exhaustion in demangle_const https://avd.aquasec.com/nvd/cve-2022-27943
libc6	CVE-2010-4756			2.36-9+deb12u13		glibc: glob implementation can cause excessive CPU and memory consumption due to... https://avd.aquasec.com/nvd/cve-2010-4756
	CVE-2018-20796					glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c https://avd.aquasec.com/nvd/cve-2018-20796
	CVE-2019-1010022					glibc: stack guard protection bypass https://avd.aquasec.com/nvd/cve-2019-1010022
	CVE-2019-1010023					glibc: running ldd on malicious ELF leads to code execution because of... https://avd.aquasec.com/nvd/cve-2019-1010023
	CVE-2019-1010024					glibc: ASLR bypass using cache of thread stack and heap https://avd.aquasec.com/nvd/cve-2019-1010024
	CVE-2019-1010025					glibc: information disclosure of heap addresses of pthread_created thread https://avd.aquasec.com/nvd/cve-2019-1010025
	CVE-2019-9192					glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c https://avd.aquasec.com/nvd/cve-2019-9192
libgcc-s1	CVE-2022-27943			12.2.0-14+deb12u1		binutils: libiberty/rust-demangle.c in GNU GCC 11.2 allows stack exhaustion in demangle_const https://avd.aquasec.com/nvd/cve-2022-27943
libgomp1						
libssl3	CVE-2025-27587			3.0.17-1~deb12u3		OpenSSL 3.0.0 through 3.3.2 on the PowerPC architecture is vulnerable https://avd.aquasec.com/nvd/cve-2025-27587
libstdc++6	CVE-2022-27943			12.2.0-14+deb12u1		binutils: libiberty/rust-demangle.c in GNU GCC 11.2 allows stack exhaustion in demangle_const https://avd.aquasec.com/nvd/cve-2022-27943

Figure 7 – Vulnerabilities within the operating system layer of the image (/tmp/juice-shop-image.tar)



Node.js (node-pkg)

Total: 55 (UNKNOWN: 0, LOW: 3, MEDIUM: 22, HIGH: 22, CRITICAL: 8)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
base64url (package.json)	NSWG-ECO-428	HIGH	fixed	0.0.6	>=3.0.0	Out-of-bounds Read https://hackerone.com/reports/321687
	GHSA-rvg8-pwq2-xj7q	MEDIUM			3.0.0	Out-of-bounds Read in base64url https://github.com/advisories/GHSA-rvg8-pwq2-xj7q
braces (package.json)	CVE-2024-4068	HIGH		2.3.2	3.0.3	braces: fails to limit the number of characters it can handle https://avd.aquasec.com/nvd/cve-2024-4068
cookie (package.json)	CVE-2024-47764	LOW		0.4.2	0.7.0	cookie: cookie accepts cookie name, path, and domain with out of bounds... https://avd.aquasec.com/nvd/cve-2024-47764
crypto-js (package.json)	CVE-2023-46233	CRITICAL		3.3.0	4.2.0	crypto-js: PBKDF2 1,000 times weaker than specified in 1993 and 1.3M times... https://avd.aquasec.com/nvd/cve-2023-46233
engine.io (package.json)	CVE-2022-41940	MEDIUM		4.1.2	3.6.1, 6.2.1	engine.io: Specially crafted HTTP request can trigger an uncaught exception https://avd.aquasec.com/nvd/cve-2022-41940
express-jwt (package.json)	CVE-2020-15084	HIGH		0.1.3	6.0.0	Authorization bypass in express-jwt https://avd.aquasec.com/nvd/cve-2020-15084
got (package.json)	CVE-2022-33987	MEDIUM		8.3.2	12.1.0, 11.8.5	nodejs-got: missing verification of requested URLs allows redirects to UNIX sockets https://avd.aquasec.com/nvd/cve-2022-33987
http-cache-semantics (package.json)	CVE-2022-25881	HIGH		3.8.1	4.1.1	http-cache-semantics: Regular Expression Denial of Service (ReDoS) vulnerability https://avd.aquasec.com/nvd/cve-2022-25881
ip (package.json)	CVE-2024-29415		affected	2.0.1		node-ip: Incomplete fix for CVE-2023-42282 https://avd.aquasec.com/nvd/cve-2024-29415
js-yaml (package.json)	CVE-2025-64718	MEDIUM	fixed	3.14.1	4.1.1, 3.14.2	js-yaml is a JavaScript YAML parser and dumper. In js-yaml 4.1.0 and... https://avd.aquasec.com/nvd/cve-2025-64718
jsonwebtoken (package.json)	CVE-2015-9235	CRITICAL		0.1.0	4.2.2	nodejs-jsonwebtoken: verification step bypass with an altered token https://avd.aquasec.com/nvd/cve-2015-9235
	CVE-2022-23539	HIGH			9.0.0	jsonwebtoken: Unrestricted key type could lead to legacy keys usagen https://avd.aquasec.com/nvd/cve-2022-23539
	NSWG-ECO-17				>=4.2.2	Verification Bypass
	CVE-2022-23540	MEDIUM			9.0.0	jsonwebtoken: Insecure default algorithm in jwt.verify() could lead to signature validation bypass... https://avd.aquasec.com/nvd/cve-2022-23540
	CVE-2022-23541					jsonwebtoken: Insecure implementation of key retrieval function could lead to Forgeable Public/Private... https://avd.aquasec.com/nvd/cve-2022-23541
	CVE-2015-9235	CRITICAL		0.4.0	4.2.2	nodejs-jsonwebtoken: verification step bypass with an altered token https://avd.aquasec.com/nvd/cve-2015-9235

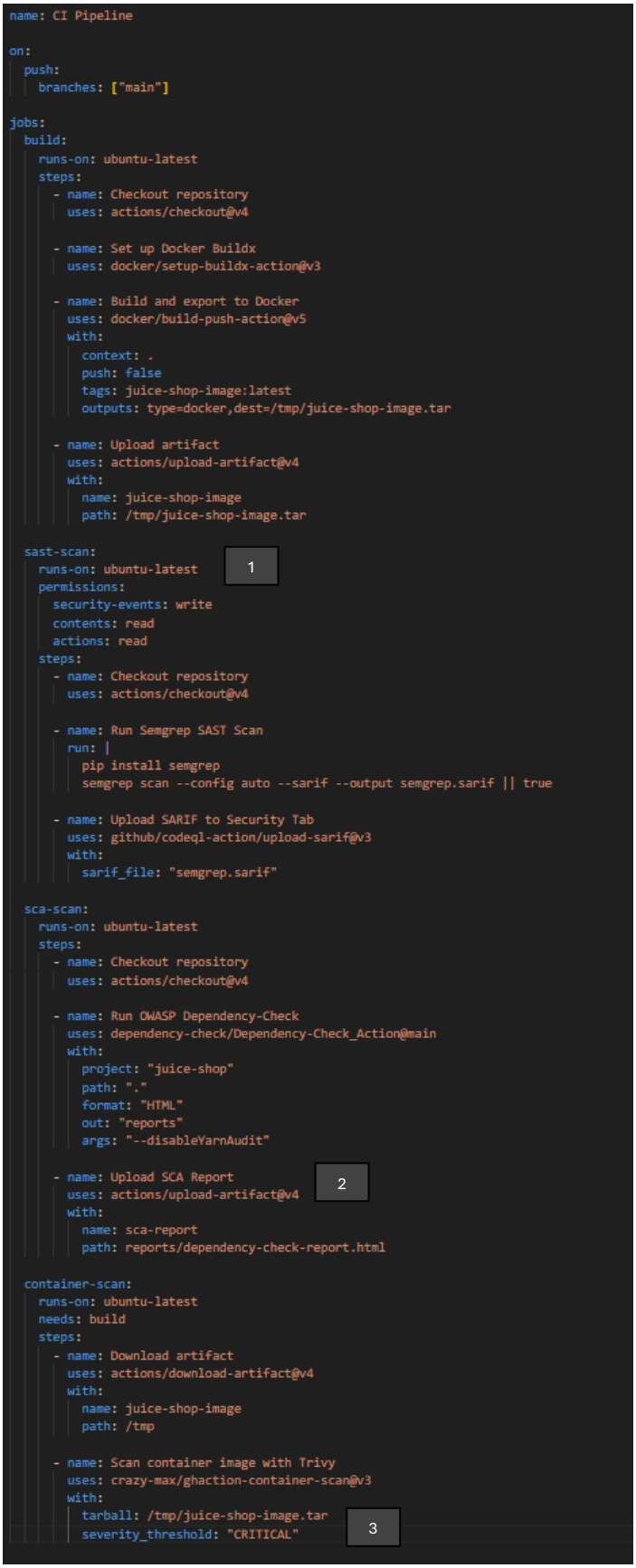
Figure 8 – vulnerabilities within the application's Node.js dependencie



Figure 9 – sensitive exposed secrets within the container

This large number of vulnerabilities shows that the pipeline is correctly configured. By successfully analyse the “/tmp/juice-shop-image.tar” shows that the pipeline could successfully built the image, passed it to the scanner, and generated an accurate report.

Phase 4:




```yaml
name: CI Pipeline

on:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and export to Docker
        uses: docker/build-push-action@v5
        with:
          context: .
          push: false
          tags: juice-shop-image:latest
          outputs: type=docker,dest=/tmp/juice-shop-image.tar

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: juice-shop-image
          path: /tmp/juice-shop-image.tar

  sast-scan:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      contents: read
      actions: read
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run Semgrep SAST Scan
        run: |
          pip install semgrep
          semgrep scan --config auto --sarif --output semgrep.sarif || true

      - name: Upload SARIF to Security Tab
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: "semgrep.sarif"

  sca-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: "juice-shop"
          path: "."
          format: "HTML"
          out: "reports"
          args: "--disableYarnAudit"

      - name: Upload SCA Report
        uses: actions/upload-artifact@v4
        with:
          name: sca-report
          path: reports/dependency-check-report.html

  container-scan:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v4
        with:
          name: juice-shop-image
          path: /tmp

      - name: Scan container image with Trivy
        uses: crazy-max/ghaction-container-scan@v3
        with:
          tarball: /tmp/juice-shop-image.tar
          severity_threshold: "CRITICAL"
```

Figure 10 – Fourth ci.yml

In Phase 4 the main changes were:

1 – sast-scan job: This job changed in order to integrate directly with GitHub's security features. First of all, the permissions were added, granting the job authorization to post findings directly to the repository's Security tab. Now instead of using a pre-packaged action, it directly installs and runs Semgrep via command line “pip install semgrep”. Finally, it generates a report in the SARIF format, which is essential for automatically populating the Code scanning alerts section under the repository's Security tab.

2 – sca-scan job: The main addition to this job is a final step to upload the generated HTML report as a workflow artifact named sca-report.

3 - container-scan job: This job now enforces an active security policy. The crucial change is the addition of the “severity_threshold: "CRITICAL"” parameter to the Trivy scanner. This configures the job to fail immediately if it detects any vulnerabilities with a "CRITICAL" severity rating in the container image.

After the pushed changes this were the results:

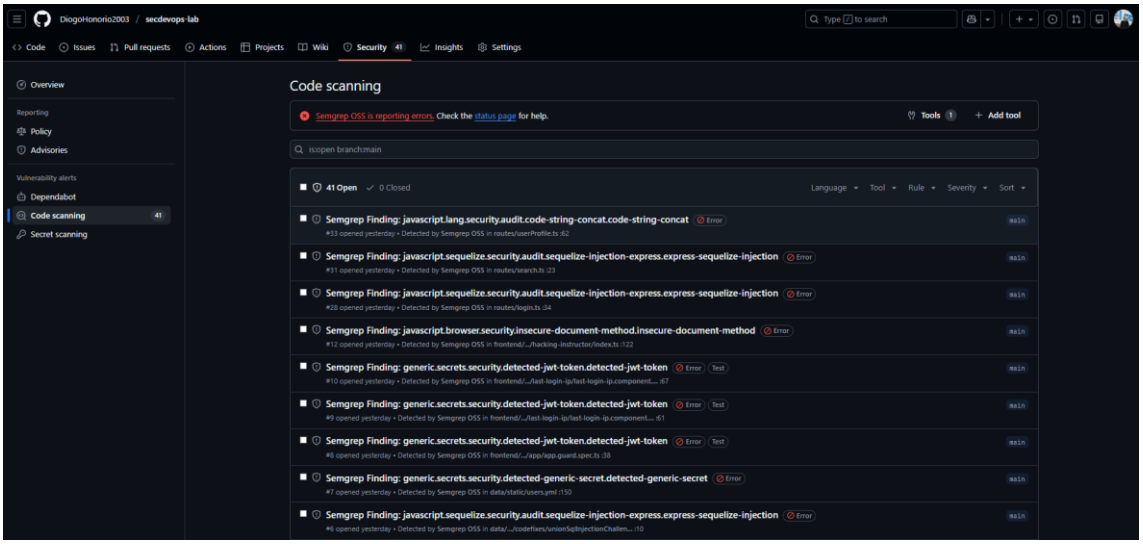


Figure 11 – Code scanning alerts in Security tab. (Semgrep)

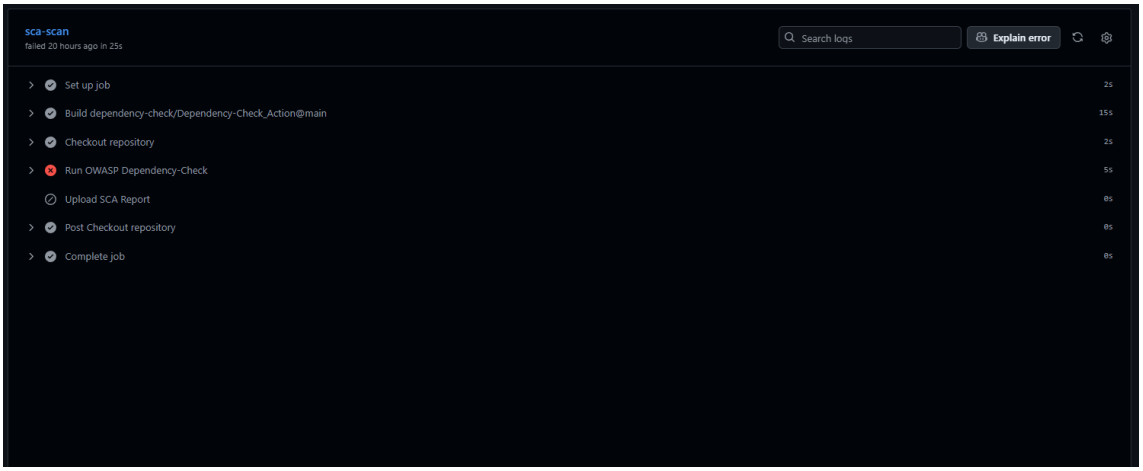


Figure 12 – Fail in generating the SCA Report

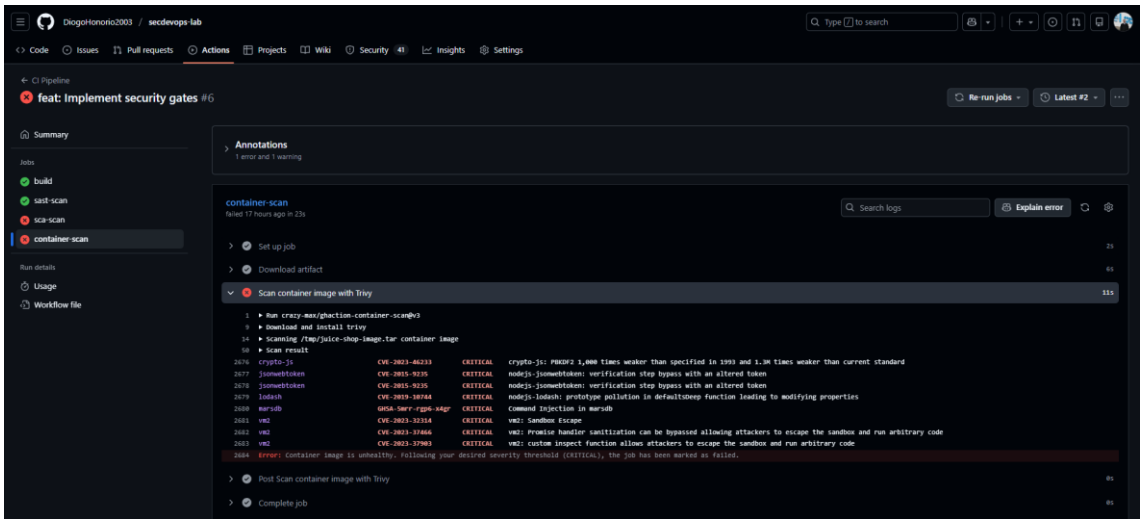


Figure 13 – Fail of container-scan due to “CRITICAL” errors

As we can see, the expected happened the sast-scan generated a report in the SARIF format, enabling the code scanning alerts to appear in the security tab and there was a fail in the container-scan due to the “CRITICAL” errors. However the SCA report was not generated.

Now it will be done an analysis for each gate, taking into consideration their outputs:

SAST Gate:
The SAST Gate's primary function is to identify security vulnerabilities and code quality issues directly within the source files. Analyzing the output confirms it was successful in this task, as all detected errors and warnings are clearly listed in the repository's Security tab, as evidenced in Figure 11.

I consider this gate essential for the workflow because it is seamlessly integrated into the GitHub interface, making security findings directly visible where developers already work, following a Shift-left principle. The only aspect I would note as suboptimal is the configuration's use of “|| true”. This clause ensures the scanning step itself never fails, which means the overall build will not stop even if critical vulnerabilities are found. In a real-world CI/CD scenario, I would define a severity threshold policy. for instance, configuring the gate to fail the build if issues classified as CRITICAL or HIGH are detected. This adjustment would transform it from a passive reporting tool into an active security enforcer, better balancing early feedback with the imperative to prevent severely vulnerable code from progressing.

SCA Gate:

This gate is responsible for searching for vulnerabilities in project's third-party libraries and dependencies. The job failed due to the error identified in the phase 2, never generating the SCA report. This gate demonstrates a crucial security control by automatically blocking the pipeline when vulnerable dependencies are found. This forces the development team to address these risks before proceeding.

Container Gate:

The Container Gate performs the final and most comprehensive security check on the built container image. Its explicit function is to fail the entire workflow if it detects any vulnerabilities with a severity classified as CRITICAL. The observed results align perfectly with this objective: the Trivy scanner identified 8 “CRITICAL” vulnerabilities, which directly caused the gate to fail, as expected.

In my opinion, this is the most critical gate in the entire pipeline. It acts as the definitive last line of defense, preventing a potentially compromised artifact from progressing toward deployment. The value of this check in a real-world scenario cannot be overstated. Furthermore, the Trivy scanner proves exceptionally useful because it doesn't just cause a failure; it provides a detailed output about the error, as seen in Figure 13. This detailed output is essential as it enables developers to quickly understand, triage, and remediate the issues, transforming a simple blocker into a powerful tool for secure development.